

Ejercicio 1 - Cambio contraseña

Cambio contraseña

```
import { useState } from "react"
import EncabezadoContrasenia from "../encabezados"
import CampoContrasenia from "../campoContraseña"
import BotonCambio from "../botonCambio"

function CambioContrasenia() {

  const [contraseniaActual, setContraseniaActual] = useState("")
  const [nuevaContrasenia, setNuevaContrasenia] = useState("")
  const [confirmarContrasenia, setConfirmarContrasenia] = useState("")

  const [errorActual, setErrorActual] = useState("")
  const [errorNueva, setErrorNueva] = useState("")
  const [errorConfirmar, setErrorConfirmar] = useState("")

  const [verActual, setVerActual] = useState(false)
  const [verNueva, setVerNueva] = useState(false)
  const [verConfirmar, setVerConfirmar] = useState(false)

  const manejarCambio = (e) => {

    e.preventDefault()

    setErrorActual("")
    setErrorNueva("")
    setErrorConfirmar("")

    if (contraseniaActual === "") {
      setErrorActual("Este campo es obligatorio")
    }

    if (nuevaContrasenia === "") {
      setErrorNueva("Este campo es obligatorio")
    }

    if (confirmarContrasenia === "") {
      setErrorConfirmar("Este campo es obligatorio")
    }

    if (
      contraseniaActual === "" ||
```

```

        nuevaContrasenia === "" ||
        confirmarContrasenia === ""
    ) {
        return
    }

    if (nuevaContrasenia.length < 8) {
        setErrorNueva("La contraseña debe tener al menos 8 caracteres")
        return
    }

    if (!/[A-Z]/.test(nuevaContrasenia)) {
        setErrorNueva("La contraseña debe contener mayúsculas")
        return
    }

    if (!/[a-z]/.test(nuevaContrasenia)) {
        setErrorNueva("La contraseña debe contener minúsculas")
        return
    }

    if (!/[0-9]/.test(nuevaContrasenia)) {
        setErrorNueva("La contraseña debe contener números")
        return
    }

    if (!/[!@#$%^&*]/.test(nuevaContrasenia)) {
        setErrorNueva("La contraseña debe contener al menos un símbolo")
        return
    }

    if (nuevaContrasenia !== confirmarContrasenia) {
        setErrorConfirmar("Las contraseñas no coinciden")
        return
    }

    alert("Contraseña cambiada correctamente")
}

return (
    <div
        className="min-vh-100 w-100 d-flex align-items-center justify-content-center p-3"
        style={{
            backgroundColor: "#0a111e",
            minWidth: "100vw"
        }}
    >

```

```
<div
  className="card shadow-lg text-white border-0"
  style={{
    maxWidth: "450px",
    width: "100%",
    backgroundColor: "#182232",
    borderRadius: "12px",
    boxShadow: "0 0 20px rgba(0, 210, 255, 0.15)"
  }}
>
```

```
<div className="card-body p-4">
```

```
<EncabezadoContrasenia />
```

```
<form onSubmit={manejarCambio}>
```

```
<CampoContrasenia
  label="Ingrese su contraseña actual"
  valor={contraseniaActual}
  setValor={setContraseniaActual}
  error={errorActual}
  setError={setErrorActual}
  ver={verActual}
  setVer={setVerActual}
/>
```

```
<CampoContrasenia
  label="Ingrese su contraseña nueva"
  valor={nuevaContrasenia}
  setValor={setNuevaContrasenia}
  error={errorNueva}
  setError={setErrorNueva}
  ver={verNueva}
  setVer={setVerNueva}
/>
```

```
<CampoContrasenia
  label="Confirmar contraseña"
  valor={confirmarContrasenia}
  setValor={setConfirmarContrasenia}
  error={errorConfirmar}
  setError={setErrorConfirmar}
  ver={verConfirmar}
  setVer={setVerConfirmar}
/>
```

```

        <BotonCambio />

      </form>

    </div>

  </div>

</div>
)
}

export default CambioContrasenia

```

Encabezados

```

function EncabezadoContrasenia() {
  return (
    <div className="text-center mb-4">
      <h2
        className="fw-bold"
        style={{
          color: "#00d2ff",
          letterSpacing: "1px"
        }}
      >
        V-MAX
      </h2>

      <h5 className="text-light">
        Cambio de contraseña
      </h5>
    </div>
  )
}

export default EncabezadoContrasenia

```

Campo Contraseña

```

function CampoContrasenia({
  label,
  valor,
  setValor,
  error,
  setError,
  ver,
  setVer

```

```

    }) {
      return (
        <div className="mb-3">

          <label className="form-label text-light">
            {label}
          </label>

          <div className="position-relative">

            <input
              type={ver ? "text" : "password"}
              className={`form-control bg-dark text-white border-secondary pe-5 ${
                error ? "is-invalid" : ""
              }}`
              value={valor}
              onChange={(e) => {
                setValor(e.target.value)
                setError("")
              }}
              style={{
                backgroundColor: "#0f172a",
                borderColor: "#334155",
                backgroundImage: "none"
              }}
            />

            <button
              type="button"
              className="btn border-0 p-0 position-absolute top-50 end-0 translate-middle-y
me-3 bg-transparent text-secondary z-3"
              onClick={() => setVer(!ver)}
            >
              {ver ? (
                <i className="bi bi-eye-slash"></i>
              ) : (
                <i className="bi bi-eye"></i>
              )}
            </button>

          </div>

          {error && (
            <div className="text-danger small mt-1">
              {error}
            </div>
          )}
        )
      )
    }
  )
}

```

```

    </div>
  )
}

```

export default CampoContrasenia

Boton Cambio

```

function BotonCambio() {
  return (
    <button
      type="submit"
      className="btn w-100 fw-bold mt-3 text-dark"
      style={{
        backgroundColor: "#00d2ff",
        borderColor: "#00d2ff",
        boxShadow: "0 0 10px rgba(0, 210, 255, 0.4)"
      }}
    >
      Cambiar contraseña
    </button>
  )
}

```

export default BotonCambio

Ejercicio 2 - Peluqueria Canina

Formulario Mascotas

```
import { useState } from "react";
```

```

function FormularioMascota({ onMascotaRegistrada }) {
  const [mascota, setMascota] = useState({
    nombre: "",
    fechaNacimiento: "",
    raza: "",
    foto: "",
    telefono: "",
    bozal: false,
    golosinas: false,
  });

```

```

  const [mensaje, setMensaje] = useState("");

```

```

const calcularEdad = (fechaNacimiento) => {
  if (!fechaNacimiento) return "";

  const nacimiento = new Date(fechaNacimiento);
  const hoy = new Date();

  let edad = hoy.getFullYear() - nacimiento.getFullYear();

  const mes = hoy.getMonth() - nacimiento.getMonth();

  if (
    mes < 0 ||
    (mes === 0 && hoy.getDate() < nacimiento.getDate())
  ) {
    edad--;
  }

  return edad;
};

```

```

const handleChange = (e) => {
  const { name, value, type, checked, files } = e.target;

  if (type === "checkbox") {
    setMascota({
      ...mascota,
      [name]: checked,
    });
  } else if (type === "file") {
    const archivo = files[0];

    setMascota({
      ...mascota,
      foto: archivo ? archivo.name : "",
    });
  } else {
    setMascota({
      ...mascota,
      [name]: value,
    });
  }
};

```

```

const handleSubmit = (e) => {
  e.preventDefault();

  if (
    !mascota.nombre ||

```

```

    !mascota.fechaNacimiento ||
    !mascota.raza ||
    !mascota.telefono
  ) {
    setMensaje("Completa todos los campos obligatorios.");
    return;
  }

  const mascotaCompleta = {
    ...mascota,
    edad: calcularEdad(mascota.fechaNacimiento),
  };

  onMascotaRegistrada(mascotaCompleta);

  setMensaje("Mascota registrada correctamente.");

  setMascota({
    nombre: "",
    fechaNacimiento: "",
    raza: "",
    foto: "",
    telefono: "",
    bozal: false,
    golosinas: false,
  });
};

return (
  <form className="formulario" onSubmit={handleSubmit}>
    <div className="form-header">
      <h2>Alta de mascota</h2>
      <p>Registra los datos de la mascota</p>
    </div>

    <div className="campo">
      <label htmlFor="nombre">Nombre</label>

      <input
        type="text"
        id="nombre"
        name="nombre"
        value={mascota.nombre}
        onChange={handleChange}
        placeholder="Ej: Toby"
      />
    </div>
  </form>
);

```

```

<div className="fila">
  <div className="campo">
    <label htmlFor="fechaNacimiento">
      Fecha de nacimiento
    </label>

    <input
      type="date"
      id="fechaNacimiento"
      name="fechaNacimiento"
      value={mascota.fechaNacimiento}
      onChange={handleChange}
    />
  </div>

  <div className="campo">
    <label htmlFor="edad">Edad</label>

    <input
      type="text"
      id="edad"
      value={
        mascota.fechaNacimiento
        ? `${calcularEdad(mascota.fechaNacimiento)} años`
        : ""
      }
      placeholder="Se calcula automáticamente"
      readOnly
    />
  </div>
</div>

<div className="campo">
  <label htmlFor="raza">Raza</label>

  <input
    type="text"
    id="raza"
    name="raza"
    value={mascota.raza}
    onChange={handleChange}
    placeholder="Ej: Caniche"
  />
</div>

<div className="campo">
  <label htmlFor="foto">Foto</label>

```

```
<input
  type="file"
  id="foto"
  name="foto"
  accept="image/*"
  onChange={handleChange}
/>
```

```
{mascota.foto && (
  <small>Archivo seleccionado: {mascota.foto}</small>
)}
</div>
```

```
<div className="campo">
  <label htmlFor="telefono">Teléfono de contacto</label>
```

```
  <input
    type="tel"
    id="telefono"
    name="telefono"
    value={mascota.telefono}
    onChange={handleChange}
    placeholder="Ej: 387 555 1234"
  />
```

```
</div>
```

```
<div className="opciones">
  <label className="checkbox">
    <input
      type="checkbox"
      name="bozal"
      checked={mascota.bozal}
      onChange={handleChange}
    />
```

```
    <span>Necesita bozal</span>
  </label>
```

```
  <label className="checkbox">
    <input
      type="checkbox"
      name="golosinas"
      checked={mascota.golosinas}
      onChange={handleChange}
    />
```

```
    <span>Puede consumir golosinas</span>
  </label>
```

```

    </div>

    <button type="submit" className="btn-registrar">
      Registrar mascota
    </button>

    {mensaje && (
      <p className="mensaje">{mensaje}</p>
    )}
  </form>
);
}

export default FormularioMascota;

```

Resumen Mascotas

Ejercicio 3 - Renderización de televisores

```

import { useState } from "react";
function ResumenMascota({ mascota }) {
  if (!mascota) {
    return (
      <div className="resumen vacio">
        <h2>Resumen</h2>
        <p>
          Todavía no se registró ninguna mascota.
        </p>
      </div>
    );
  }

  return (
    <div className="resumen">
      <div className="resumen-header">
        <span className="icono">🐾</span>

        <div>
          <h2>{mascota.nombre}</h2>
          <p>Registro realizado correctamente</p>
        </div>
      </div>
    </div>
  );
}

```

```

<div className="datos">
  <div className="dato">
    <span>Raza</span>
    <strong>{mascota.raza}</strong>
  </div>

  <div className="dato">
    <span>Edad</span>
    <strong>{mascota.edad} años</strong>
  </div>

  <div className="dato">
    <span>Teléfono</span>
    <strong>{mascota.telefono}</strong>
  </div>

  <div className="dato">
    <span>Bozal</span>
    <strong>
      {mascota.bozal ? "Sí" : "No"}
    </strong>
  </div>

  <div className="dato">
    <span>Golosinas</span>
    <strong>
      {mascota.golosinas ? "Sí" : "No"}
    </strong>
  </div>

  <div className="dato">
    <span>Foto</span>
    <strong>
      {mascota.foto || "No seleccionada"}
    </strong>
  </div>
</div>
);
}

```

```
export default ResumenMascota;
```

```

const televisores = [
  { id: 1, marca: "Samsung", pulgadas: 55, sistema_operativo: "Tizen", wifi: false },
  { id: 2, marca: "LG", pulgadas: 65, sistema_operativo: "webOS", wifi: false },
  { id: 3, marca: "Sony", pulgadas: 50, sistema_operativo: "Google TV", wifi: false },
  { id: 4, marca: "TCL", pulgadas: 43, sistema_operativo: "Android TV", wifi: false },

```

```

    { id: 5, marca: "Hisense", pulgadas: 32, sistema_operativo: "VIDAA", wifi: false },
    { id: 6, marca: "Philips", pulgadas: 58, sistema_operativo: "SAPHI", wifi: false },
    { id: 7, marca: "Xiaomi", pulgadas: 43, sistema_operativo: "Android TV", wifi: false },
    { id: 8, marca: "Samsung", pulgadas: 75, sistema_operativo: "Tizen", wifi: true },
    { id: 9, marca: "LG", pulgadas: 32, sistema_operativo: "webOS", wifi: true },
    { id: 10, marca: "Panasonic", pulgadas: 55, sistema_operativo: "My Home Screen", wifi:
false },
    { id: 11, marca: "TCL", pulgadas: 65, sistema_operativo: "Roku TV", wifi: true },
    { id: 12, marca: "Sony", pulgadas: 85, sistema_operativo: "Google TV", wifi: true },
    { id: 13, marca: "Hisense", pulgadas: 50, sistema_operativo: "Google TV", wifi: true },
    { id: 14, marca: "Philips", pulgadas: 65, sistema_operativo: "Android TV", wifi: true },
    { id: 15, marca: "BGH", pulgadas: 43, sistema_operativo: "Android TV", wifi: false },
    { id: 16, marca: "Noblex", pulgadas: 32, sistema_operativo: "VIDAA", wifi: false },
    { id: 17, marca: "RCA", pulgadas: 40, sistema_operativo: "Android TV", wifi: false },
    { id: 18, marca: "Samsung", pulgadas: 43, sistema_operativo: "Tizen", wifi: true },
    { id: 19, marca: "LG", pulgadas: 55, sistema_operativo: "webOS", wifi: true },
    { id: 20, marca: "Xiaomi", pulgadas: 55, sistema_operativo: "Google TV", wifi: true },
    { id: 21, marca: "Sony", pulgadas: 65, sistema_operativo: "Google TV", wifi: true },
    { id: 22, marca: "TCL", pulgadas: 50, sistema_operativo: "Google TV", wifi: true },
    { id: 23, marca: "Hisense", pulgadas: 43, sistema_operativo: "VIDAA", wifi: true },
    { id: 24, marca: "Panasonic", pulgadas: 43, sistema_operativo: "Android TV", wifi: true },
    { id: 25, marca: "Hitachi", pulgadas: 32, sistema_operativo: "Android TV", wifi: false },
    { id: 26, marca: "Samsung", pulgadas: 50, sistema_operativo: "Tizen", wifi: true },
    { id: 27, marca: "LG", pulgadas: 77, sistema_operativo: "webOS", wifi: true },
    { id: 28, marca: "Philips", pulgadas: 50, sistema_operativo: "Google TV", wifi: true },
    { id: 29, marca: "TCL", pulgadas: 32, sistema_operativo: "Android TV", wifi: true },
    { id: 30, marca: "Sony", pulgadas: 55, sistema_operativo: "Google TV", wifi: true },
    { id: 31, marca: "Hisense", pulgadas: 75, sistema_operativo: "VIDAA", wifi: true },
    { id: 32, marca: "Noblex", pulgadas: 50, sistema_operativo: "Google TV", wifi: true },
    { id: 33, marca: "BGH", pulgadas: 55, sistema_operativo: "Android TV", wifi: true },
    { id: 34, marca: "RCA", pulgadas: 32, sistema_operativo: "Roku TV", wifi: true },
    { id: 35, marca: "Xiaomi", pulgadas: 65, sistema_operativo: "Google TV", wifi: true },
    { id: 36, marca: "Samsung", pulgadas: 65, sistema_operativo: "Tizen", wifi: true },
    { id: 37, marca: "LG", pulgadas: 43, sistema_operativo: "webOS", wifi: true },
    { id: 38, marca: "Sharp", pulgadas: 50, sistema_operativo: "Android TV", wifi: false },
    { id: 39, marca: "JVC", pulgadas: 32, sistema_operativo: "Linux OS", wifi: false },
    { id: 40, marca: "TCL", pulgadas: 75, sistema_operativo: "Google TV", wifi: true },
    { id: 41, marca: "Hisense", pulgadas: 65, sistema_operativo: "Google TV", wifi: true },
    { id: 42, marca: "Philips", pulgadas: 75, sistema_operativo: "Android TV", wifi: true },
    { id: 43, marca: "Sony", pulgadas: 75, sistema_operativo: "Google TV", wifi: true },
    { id: 44, marca: "Samsung", pulgadas: 85, sistema_operativo: "Tizen", wifi: true },
    { id: 45, marca: "LG", pulgadas: 83, sistema_operativo: "webOS", wifi: true },
    { id: 46, marca: "Noblex", pulgadas: 43, sistema_operativo: "Android TV", wifi: true },
    { id: 47, marca: "BGH", pulgadas: 65, sistema_operativo: "Google TV", wifi: true },
    { id: 48, marca: "Hitachi", pulgadas: 50, sistema_operativo: "Android TV", wifi: true },
    { id: 49, marca: "Xiaomi", pulgadas: 75, sistema_operativo: "Google TV", wifi: true },
    { id: 50, marca: "Hyundai", pulgadas: 40, sistema_operativo: "Android TV", wifi: false },
];

```

```

function CatalogoTelevisores() {
  const [seccionActiva, setSeccionActiva] = useState("sin_wifi");

  return (
    <div className="w-100 py-5" style={{ backgroundColor: "#0f172a", minHeight: "100vh" }}>
      <div className="container">

        <div className="text-center mb-5">
          <span className="badge px-3 py-2 rounded-pill text-uppercase mb-2" style={{
            backgroundColor: "#0369a1", color: "#e0f2fe", fontWeight: "600" }}>
            Smart TV Store
          </span>
          <h1 className="fw-bold display-5 text-white">Catálogo de Smart TVs</h1>
        </div>

        <div className="d-flex justify-content-center mb-5">
          <div className="p-1 rounded-4 d-inline-flex flex-wrap justify-content-center gap-1
            shadow" style={{ backgroundColor: "#1e293b", border: "1px solid #334155" }}>
            <button
              className={`btn btn-sm px-4 py-2 rounded-3 fw-semibold border-0 ${
                seccionActiva === "sin_wifi" ? "text-white shadow-sm" : "text-secondary"
              }`}
              style={{
                backgroundColor: seccionActiva === "sin_wifi" ? "#0284c7" : "transparent",
                transition: "all 0.2s ease"
              }}
              onClick={() => setSeccionActiva("sin_wifi")}
            >
              Sin WiFi
            </button>

            <button
              className={`btn btn-sm px-4 py-2 rounded-3 fw-semibold border-0 ${
                seccionActiva === "google_tv" ? "text-white shadow-sm" : "text-secondary"
              }`}
              style={{
                backgroundColor: seccionActiva === "google_tv" ? "#0284c7" :
                "transparent", transition: "all 0.2s ease"
              }}
              onClick={() => setSeccionActiva("google_tv")}
            >
              Google TV
            </button>

            <button
              className={`btn btn-sm px-4 py-2 rounded-3 fw-semibold border-0 ${

```

```

        seccionActiva === "samsung_wifi" ? "text-white shadow-sm" :
"text-secondary"
      }}
      style={{
        backgroundColor: seccionActiva === "samsung_wifi" ? "#0284c7" :
"transparent", transition: "all 0.2s ease"
      }}
      onClick={() => setSeccionActiva("samsung_wifi")}
    >
      Samsung + WiFi
    </button>
  </div>
</div>

<div className="row g-4">
  {televisores.map((tv) => {

    if (seccionActiva === "sin_wifi" && tv.wifi === true) {
      return null;
    }

    if (seccionActiva === "google_tv" && tv.sistema_operativo !== "Google TV") {
      return null;
    }

    if (seccionActiva === "samsung_wifi" && (tv.marca !== "Samsung" || tv.wifi ===
false)) {
      return null;
    }

    return (
      <div key={tv.id} className="col-12 col-sm-6 col-md-4 col-lg-3">
        <div className="card h-100 rounded-4 overflow-hidden shadow" style={{
backgroundColor: "#1e293b", border: "1px solid #334155" }}>

          <div className="px-4 pt-4 pb-3 d-flex justify-content-between
align-items-center" style={{ backgroundColor: "#0f172a", borderBottom: "1px solid #334155"
}}>
            <span className="fw-bold fs-5 text-white">{tv.marca}</span>
            <span className="badge rounded-pill fs-6 px-3 py-1" style={{
backgroundColor: "#312e81", color: "#e0e7ff", fontWeight: "600" }}>
              {tv.pulgadas}"
            </span>
          </div>

          <div className="card-body p-4 d-flex flex-column justify-content-between">
            <div className="mb-3">

```

```

        <div className="d-flex align-items-center mb-2">
          <span className="text-secondary small me-2">Sistema:</span>
          <span className="fw-semibold
text-light">{tv.sistema_operativo}</span>
        </div>
      </div>

      <div className="pt-3 border-top d-flex justify-content-between
align-items-center" style={{ borderColor: "#334155" }}>
        <span className="small text-secondary">Conectividad:</span>
        {tv.wifi ? (
          <span className="badge rounded-pill px-3 py-1" style={{
backgroundColor: "#14532d", color: "#bbf7d0", border: "1px solid #166534" }}>
            Con WiFi
          </span>
        ) : (
          <span className="badge rounded-pill px-3 py-1" style={{
backgroundColor: "#7f1d1d", color: "#fecaca", border: "1px solid #991b1b" }}>
            Sin WiFi
          </span>
        )}
      </div>
    </div>
  </div>
);
}}
</div>

</div>
</div>
);
}

```

```

export default CatalogoTelevisores;

```